Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Data Preparation

In this workshop the churn problem is approached from a different angle. Instead of summarising each subscriber into a single tabular row, as in Workshop 1, we use the dates in the dataset to treat churn prediction as a **sequence problem**: each subscriber is represented by the day-by-day evolution of their listening activity and subscription status. This requires transforming the raw data in a different way from the previous workshop. The pipeline is implemented in `workshops/w2/w2_sequence_preprocessing.ipynb` and the modules under `workshops/w2/src/data_processing/`.

1.1 Structure of the Raw Data

The dataset was obtained from the *WSDM–KKBox's Churn Prediction Challenge*, hosted on Kaggle in 2017 with data donated by KKBOX [2]. It covers approximately 2015 to 2017 and contains the history of each subscriber's transactions and music listening logs. Three of the raw tables are used to build the sequences: the labelled cohorts (Table 1), the daily listening logs (Table 2) and the payment transactions (Table 3). The member demographics table is not used in this workshop.

Feature	Description	Data type
msno	Hashed subscriber identifier (primary key)	`str`
is_churn	Churn label: 1 if the user did not renew within 30 days of expiry, 0 otherwise	`int8`

Table 1: Schema of `train.csv` (February cohort) and `train_v2.csv` (March cohort).

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
date	Listening date in YYYYMMDD format	`int32`
num_25	Number of songs played less than 25% of their duration	`int32`
num_50	Number of songs played between 25% and 50% of their duration	`int32`
num_75	Number of songs played between 50% and 75% of their duration	`int32`
num_985	Number of songs played between 75% and 98.5% of their duration	`int32`
num_100	Number of songs played at least 98.5% of their duration	`int32`
num_unq	Number of unique songs played on that day	`int32`
total_secs	Total seconds of music listened to on that day	`float64`

Table 2: Schema of `user_logs.csv` and `user_logs_v2.csv`.

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
payment_method_id	Payment method encoded as an integer code	`int8`
payment_plan_days	Number of days in the purchased subscription plan	`int16`
plan_list_price	Listed price of the plan in New Taiwan Dollars (NTD)	`int16`
actual_amount_paid	Amount actually paid by the subscriber in NTD	`int16`
is_auto_renew	Automatic renewal flag (1 = yes, 0 = no)	`int8`
transaction_date	Date of the transaction in YYYYMMDD format	`int32`
membership_expire_date	Membership expiration date after the transaction in YYYYMMDD format	`int32`
is_cancel	Cancellation flag (1 = yes, 0 = no)	`int8`

Table 3: Schema of `transactions.csv` and `transactions_v2.csv`.

1.2 Tensor Construction

1.2.1 Temporal Split and Window Overlap

Each cohort c is defined by a cutoff date τ_c, the last day whose information may be used. Its sequence window contains the $L = 90$ consecutive calendar days that end on the cutoff:

$$W_c = \{\, \tau_c - 89,\ \tau_c - 88,\ \ldots,\ \tau_c \,\}, \qquad |W_c| = L = 90. \tag{1}$$

A calendar date $d \in W_c$ is mapped to the day index

$$t(d) = d - (\tau_c - 89) \in \{0, 1, \ldots, 89\}, \tag{2}$$

so $t = 0$ is the oldest day of the window and $t = 89$ is the cutoff. Table 4 lists the two cohorts.

Cohort	**Role**	**Window W_c**	**Target**	**Users N_c**	**Churn rate**
`train_feb`	Training	2016-11-03 to 2017-01-31	February 2017	992,931	6.4%
`test`	Test	2016-12-01 to 2017-02-28	March 2017	970,960	9.0%

Table 4: Labelled cohorts, their 90-day windows and churn targets.

We build the training tensor from each user's activity between 2016-11-03 and 2017-01-31, with February 2017 churn as the target. The test tensor uses the window from 2016-12-01 to 2017-02-28, with March 2017 churn as the target. The two windows overlap by 62 days (2016-12-01 to 2017-01-31), as shown in Figure 1.

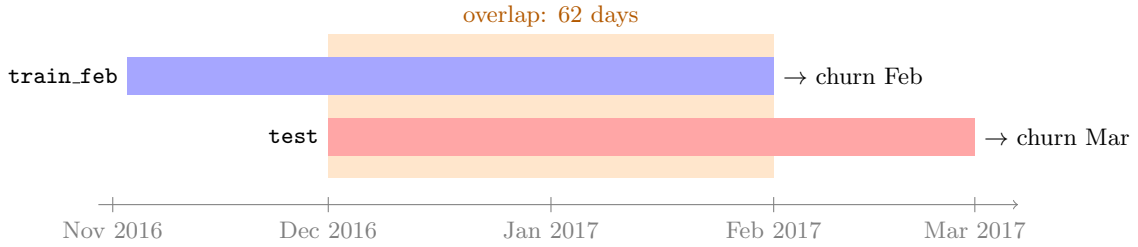


Figure 1: 90-day windows of the two cohorts. Each window ends at its cutoff (2017-01-31 and 2017-02-28) and is followed by its target month.

This overlap is intentional. It follows from using a fixed-length lookback window that advances one month at a time, which is how a subscription service would score its customers in production. Each month, the model receives the most recent 90 days of behaviour and predicts whether the customer will renew in the following month, so consecutive scoring windows necessarily share most of their history.

The overlap does not leak target information. Every feature is computed using only data up to its window's cutoff, and the labels belong to different, non-overlapping periods (February for training, March for testing). During training the model never observes the March outcome, nor any activity after 2017-01-31. The test set therefore measures forward-looking temporal generalisation: how well a model trained on one month's snapshot predicts the next month's churn, including any behavioural drift between the two months.

Many users appear in both cohorts (881,701 of the February users are also in the March cohort), as they would in real monthly scoring. A shared integer identifier `uid` is assigned over the union of both cohorts (1,082,190 unique users) and is used only to join each user with their logs and transactions. Each cohort keeps its own complete list of users, its own row order and its own labels: a user present in only one cohort appears only in that cohort's tensor, and a user present in both appears twice, with a

different cutoff, sequence and label each time. Our results should therefore be read as performance on an existing customer base rather than on entirely unseen users.

1.2.2 Cleaning

Transactions. Both transaction files are concatenated, restricted to the target users and to transactions on or before the cutoff. Exact duplicates are then removed: two rows are considered duplicates when the user (`msno`) and all eight transaction fields (`payment_method_id`, `payment_plan_days`, `plan_list_price`, `actual_amount_paid`, `is_auto_renew`, `transaction_date`, `membership_expire_date`, `is_cancel`) are equal. Only the first copy is kept. Two rows on the same day with different values are treated as different payments and both are kept.

Listening logs. The raw logs may contain several rows for the same user and day. Before aggregation, negative counters are clipped to zero and non-finite values of `total_secs` are replaced by zero. The rows of the same user and day are then summed into a single row. For example, if user A has three rows on 2017-02-01, with `num_25` equal to 3, 2 and 1, the aggregated row has `num_25` $= 6$; the other counters are summed in the same way. Formally, for user i, day d and counter k:

$$v_{i,d,k} = \sum_{r \in R_{i,d}} \max(0,\, v_{r,k}), \tag{3}$$

where $R_{i,d}$ is the set of raw log rows of user i on day d. The number of raw rows $|R_{i,d}|$ is kept to define the activity flag.

1.2.3 Listening Channels

The aggregated logs are placed into the window. Days without any log row are filled with zeros. The seven counters are transformed with $\log(1 + x)$ to compress their heavy-tailed distribution; standard scaling is fitted later, on the training split only, inside the model notebook. An eighth channel, `active`, indicates whether a log row exists for that day, independently of the counter values:

$$x_{i,t,k} = \log(1 + v_{i,d,k})\,, \quad k = 1, \ldots, 7, \qquad a_{i,t} = \mathbf{1}[\,|R_{i,d}| > 0\,]\,, \tag{4}$$

with $t = t(d)$. A day with a log row where every counter is zero is still active, which distinguishes "logged in but did not listen" from "no record". Table 5 lists the listening channels.

Index	Channel	Description	Encoding
0	num_25	Songs played less than 25%	$\log(1+x)$
1	num_50	Songs played 25–50%	$\log(1+x)$
2	num_75	Songs played 50–75%	$\log(1+x)$
3	num_985	Songs played 75–98.5%	$\log(1+x)$
4	num_100	Songs played at least 98.5%	$\log(1+x)$
5	num_unq	Unique songs played	$\log(1+x)$
6	total_secs	Seconds listened	$\log(1+x)$
7	active	1 if a log row exists for the day, 0 otherwise	flag

Table 5: Listening channels (8) of the sequence tensor.

1.2.4 Transaction Channels: Events and States

The tensor is completed with twelve transaction channels, divided into two groups:

- **Events** encode what happened on a specific day. They are written only on the days with at least one payment and are zero on every other day.
- **States** encode the subscription status known at the end of a specific day. They are carried forward to the following days until the next payment updates them.

First, the transactions of each user are aggregated per day. Event values are summed over all payments of the day. State values (cancellation, auto-renewal and expiry date) are taken from the last record of that day in source order, and a flag marks the day as ambiguous when the records of the same day disagree on any of these three values.

Payments before the window. Only the **latest payment day before the window** is kept from the earlier history; all earlier payments are summarised by that single record. This record initialises the state at $t = 0$ but does not create an event inside the window. A payment on day t writes the event channels on day t and updates the state from day t onward. Future payments never modify earlier days, which guarantees that the tensor is causal. This property is verified in the notebook by comparing the vectorised builder against a day-by-day reference implementation and checking that adding a later payment leaves every earlier day unchanged.

Formally, let P_i be the set of payment days of user i up to the cutoff, and let

$$s_i(t) = \max\{\, p \in P_i : p \le d_t \,\} \tag{5}$$

be the most recent payment day known on day d_t (possibly before the window). The state is known

when $s_i(t)$ exists, and the recency and expiry channels are

$$\text{tx_days_since_last}_{i,t} = \log\Big(1 + \text{clip}\left(d_t - s_i(t),\, 0,\, 3650\right)\Big), \tag{6}$$

$$\text{tx_days_to_expire}_{i,t} = \text{slog}\Big(\text{clip}\left(e_i(t) - d_t,\, -3650,\, 3650\right)\Big), \tag{7}$$

where $e_i(t)$ is the membership expiry date recorded on day $s_i(t)$, and $\text{slog}(x) = \text{sign}(x)\log(1+|x|)$ is the signed logarithm, used for quantities that can be negative (an already expired membership, or a negative amount paid). When the state is not yet known, all state channels are zero.

Worked example. Consider a user of the `test` cohort, whose window starts on 2016-12-01 ($t = 0$). The user paid on 2016-11-20, before the window, with auto-renewal disabled and a membership that expires on 2016-12-20, and made no further payments. The pre-window payment initialises the state, so $s_i(t) =$ 2016-11-20 and $e_i(t) =$ 2016-12-20 for every day of the window, and no event is written on any day. Table 6 shows the two channels on three days. On day 19 the membership expires and `tx_days_to_expire` is zero; from day 20 on it is negative, a strong signal that the user has not renewed. These values were obtained by running this user through the same builder used for the export.

Day t	**Date** d_t	$d_t - s_i(t)$	**tx_days_since_last**	$e_i(t) - d_t$	**tx_days_to_expire**
0	2016-12-01	11	$\log(1+11) = 2.48$	19	$\text{slog}(19) = 3.00$
19	2016-12-20	30	$\log(1+30) = 3.43$	0	$\text{slog}(0) = 0.00$
24	2016-12-25	35	$\log(1+35) = 3.58$	-5	$\text{slog}(-5) = -1.79$

Table 6: Recency and expiry channels for a user whose only payment (2016-11-20, expiring 2016-12-20) falls before the `test` window.

Table 7 describes each transaction channel.

Index	**Channel**	**Group**	**Description**	**Encoding**
8	tx_count	Event	Number of payments on the day	$\log(1+x)$
9	tx_paid_sum	Event	Total amount paid on the day (NTD)	$\text{slog}(x)$
10	tx_plan_days_sum	Event	Total plan days purchased on the day	$\log(1+x)$
11	tx_cancel_count	Event	Number of cancellation records on the day	$\log(1+x)$
12	tx_auto_renew_count	Event	Number of auto-renew records on the day	$\log(1+x)$
13	tx_state_known	State	1 once at least one payment has been observed up to the day, including the pre-window payment	flag

Continued on next page

Index	Channel	Group	Description	Encoding
14	tx_last_is_cancel	State	Cancellation flag of the most recent payment day	flag
15	tx_last_is_auto_renew	State	Auto-renew flag of the most recent payment day	flag
16	tx_days_since_last	State	Days since the most recent payment, clipped to $[0, 3650]$	$\log(1+x)$
17	tx_days_to_expire	State	Membership expiry minus the current day, clipped to $[-3650, 3650]$	$\mathrm{slog}(x)$
18	tx_expiry_known	State	1 if the expiry date of the most recent payment is valid	flag
19	tx_state_ambiguous	State	1 if the records of the most recent payment day disagree on cancellation, auto-renewal or expiry	flag

Table 7: Transaction channels (12) of the sequence tensor.

1.2.5 Resulting Tensor

For each user i of cohort c, the 20 channels are stacked into a matrix whose rows are the 90 days, ordered from oldest to newest, and whose columns are the channels in the order of Tables 5 and 7:

$$X_i^{(c)} = \begin{bmatrix} x_{i,0,0} & x_{i,0,1} & \cdots & x_{i,0,19} \\ x_{i,1,0} & x_{i,1,1} & \cdots & x_{i,1,19} \\ \vdots & \vdots & \ddots & \vdots \\ x_{i,89,0} & x_{i,89,1} & \cdots & x_{i,89,19} \end{bmatrix} \in \mathbb{R}^{L \times F}, \qquad L = 90,\ F = 20. \tag{8}$$

Stacking the N_c users of the cohort gives the cohort tensor

$$\mathcal{X}^{(c)} = \left[\, X_1^{(c)}, X_2^{(c)}, \ldots, X_{N_c}^{(c)} \,\right] \in \mathbb{R}^{N_c \times L \times F}, \tag{9}$$

together with the label vector $y^{(c)} \in \{0,1\}^{N_c}$, aligned so that position i of $y^{(c)}$ is the label of $X_i^{(c)}$. The channel dimension splits into three blocks:

$$X_i^{(c)} = \left[\, \underbrace{X_i^{\text{listen}}}_{\mathbb{R}^{90\times 8}} \;\middle|\; \underbrace{X_i^{\text{event}}}_{\mathbb{R}^{90\times 5}} \;\middle|\; \underbrace{X_i^{\text{state}}}_{\mathbb{R}^{90\times 7}} \,\right]. \tag{10}$$

In this way we obtain $\mathcal{X}^{(\text{train_feb})} \in \mathbb{R}^{992{,}931\times 90\times 20}$ and $\mathcal{X}^{(\text{test})} \in \mathbb{R}^{970{,}960\times 90\times 20}$. The tensors are

stored as `float16` arrays (`seq_lstm_train_feb.npy` and `seq_lstm_test.npy`, about 3.5 GB each) with their labels in `labels_<cohort>.parquet`, in the directory `workshops/w2/export-sequential-v5/`. Before export, an audit rejects the tensor if any of the following fails: the expected shape, finite values, binary flags, non-negative counts and recency, zero event channels on days without a payment, and zero state channels before the state is known.

2 Recurrent Models

We compare two gated recurrent neural networks: a Long Short-Term Memory network (LSTM) and a Gated Recurrent Unit network (GRU). Both read the 90 days in chronological order and maintain a hidden representation of the customer's behaviour. Prior empirical work found GRU performance comparable to LSTM on sequence-modelling tasks, while noting that the relative result depends on the dataset and task [1]. Our comparison isolates the recurrent cell: the input tensor, preprocessing, data partitions, classifier head, loss, optimiser, search space and evaluation protocol are identical. The models are implemented in PyTorch by the shared `RecurrentClassifier` and its subclasses `LSTMModel` and `GRUModel` (`workshops/w2/src/models/recurrent_models/recurrent_classifier.py`); training and evaluation are orchestrated in `workshops/w2/w2_neural_models.ipynb`.

2.1 Architecture

The common architecture has the following layers (Figure 2):

- **Input layer.** A batch of users taken from the tensor built before, with shape $(B, 90, 20)$: B users, 90 days and 20 channels. Each day t of user i is the vector $x_{i,t} \in \mathbb{R}^{20}$.
- **Recurrent encoder.** Either stacked LSTM or stacked GRU layers. The first layer reads the 20 channels and each subsequent layer reads the hidden states of the layer below.
- **Hidden dimension** (`hidden_dim`). The number of units in each hidden state $h_t \in \mathbb{R}^H$; it controls the capacity of the temporal representation.
- **Layer dimension** (`layer_dim`). The number of stacked recurrent layers, between 2 and 4.
- **Batch normalisation layer.** After the 90 days, the output of the last layer on the last day, $h_{90} \in \mathbb{R}^H$, has shape (B, H). This vector summarises the customer's behaviour over the 90 days. `BatchNorm1d` normalises it across the batch, which stabilises the training.
- **Output layer.** A linear layer applies the transformation $z = w^\top \tilde{h}_{90} + b$ and produces one value per user, the *logit* $z \in \mathbb{R}$.

2.1.1 LSTM Cell

For each day, an LSTM cell updates a separate memory cell c_t and hidden state h_t with

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \qquad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \qquad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \tag{11}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t), \tag{12}$$

where f_t, i_t and o_t are the forget, input and output gates, σ is the sigmoid function and $\odot$ is the element-wise product. The states start at $h_0 = c_0 = 0$.

The explicit memory path allows the forget gate to retain information over many time steps, while the input and output gates control what is written and exposed. This flexibility comes at a computational cost: each layer computes four affine transformations (three gates and one candidate state).

2.1.2 GRU Cell

The GRU merges the memory and hidden states into a single vector. For each day it computes a reset gate r_t, an update gate u_t and a candidate state n_t:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r), \tag{13}$$

$$u_t = \sigma(W_u x_t + U_u h_{t-1} + b_u), \tag{14}$$

$$n_t = \tanh(W_n x_t + b_n + r_t \odot (U_n h_{t-1} + d_n)), \tag{15}$$

$$h_t = (1 - u_t) \odot n_t + u_t \odot h_{t-1}. \tag{16}$$

The reset gate controls how much previous information enters the candidate, and the update gate interpolates between the new candidate and the previous state. Thus, when u_t is close to one, the GRU preserves its previous memory; when it is close to zero, it replaces it with the candidate. A GRU uses three affine transformations instead of the LSTM's four and has no separate cell state, so it is generally smaller and faster for the same H and number of layers.

For either encoder, the states start at zero and the classifier uses only the final hidden state. The model output and churn probability are

$$z_i = w^\top \mathrm{BN}\left(h_{i,90}\right) + b, \qquad \hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}. \tag{17}$$

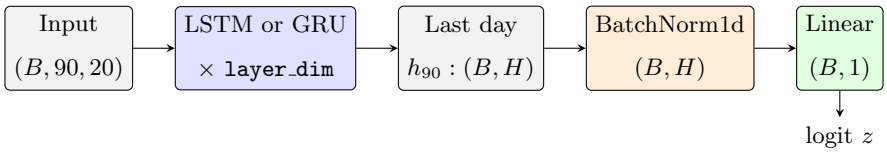


Figure 2: Common LSTM/GRU classifier architecture. Only the recurrent cell changes between models; B is the batch size and H the hidden dimension.

2.2 Loss Function and Optimiser

Since churn is a binary classification problem, we use the binary cross-entropy loss. In PyTorch it is defined as `criterion = nn.BCEWithLogitsLoss()`. This function receives the logit z_i and applies the sigmoid internally, so the model does not include a sigmoid layer. Combining both operations in one function is numerically stable: it never evaluates $\log(0)$ when the sigmoid saturates. For a batch of B users the loss is

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\Big[y_i \log \sigma(z_i) + (1-y_i)\log\big(1-\sigma(z_i)\big)\Big]. \tag{18}$$

At prediction time, where there is no loss, the probability is obtained explicitly with `torch.sigmoid(logits)`.

Backpropagation. Training needs the gradient of the loss with respect to every weight θ of the network. Backpropagation computes it by applying the chain rule backwards, from the loss to the input, through the same operations used in the forward pass (Figure 3). Using the derivative of the sigmoid, $\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$, the first two steps for one user are

$$\frac{\partial \mathcal{L}}{\partial \hat{p}} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}}, \qquad \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{p}} \cdot \sigma'(z) = \frac{\hat{p}-y}{\hat{p}(1-\hat{p})} \cdot \hat{p}(1-\hat{p}) = \hat{p} - y. \tag{19}$$

The derivative of the sigmoid cancels the denominator of the cross-entropy, so the error signal that enters the network is simply the difference between the predicted probability and the true label. It is large when the prediction is confidently wrong and close to zero when it is correct, and the saturation of the sigmoid cannot make it vanish while the prediction is wrong. This simplification is also what `BCEWithLogitsLoss` exploits to compute the loss and its gradient in a stable way. From z, the gradient continues to the output layer and to the hidden state:

$$\frac{\partial \mathcal{L}}{\partial w} = (\hat{p}-y)\,\tilde{h}_{90}, \qquad \frac{\partial \mathcal{L}}{\partial b} = \hat{p} - y, \qquad \frac{\partial \mathcal{L}}{\partial \tilde{h}_{90}} = (\hat{p}-y)\,w. \tag{20}$$

It then passes through batch normalisation to h_{90} and travels from day 90 to day 1 through the recurrent connections, a procedure called *backpropagation through time*. This outer procedure is identical for LSTM and GRU; only the local derivatives inside their gates differ. Since the recurrent weights are shared across days, their gradient is the sum of the contributions of the 90 time steps:

$$\frac{\partial \mathcal{L}}{\partial \theta_{\mathrm{RNN}}} = \sum_{t=1}^{90} \frac{\partial \mathcal{L}}{\partial h_t}\,\frac{\partial h_t}{\partial \theta_{\mathrm{RNN}}}, \qquad \theta_{\mathrm{RNN}} \in \{\theta_{\mathrm{LSTM}}, \theta_{\mathrm{GRU}}\}. \tag{21}$$

For a batch, these gradients are averaged over the B users. Finally, AdamW uses them to update every weight.

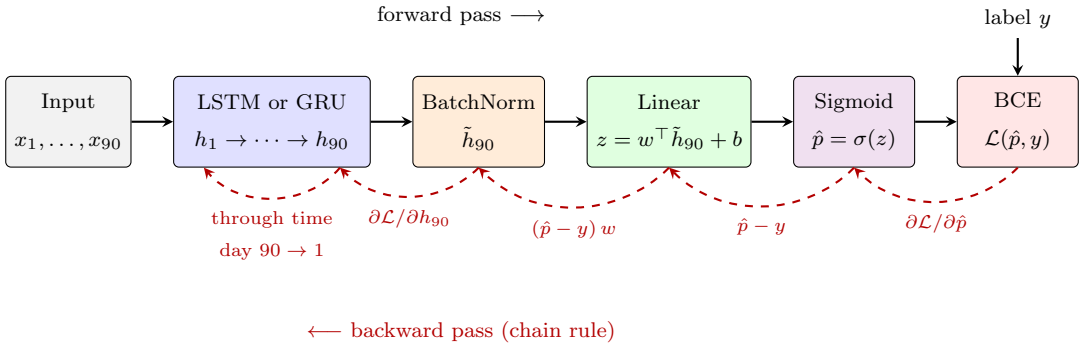


Figure 3: Forward and backward passes shared by LSTM and GRU. At the logit the gradient reduces to $\hat{p} - y$; inside the selected recurrent cell it travels back through the 90 days and its gates.

The weights are updated with the **AdamW** optimiser, which computes an adaptive step for each weight from the gradient and applies weight decay separately from that step. It receives two parameters:

- **Learning rate** (η): how large each update step is.
- **Weight decay** (λ): shrinks the weights towards zero at every step, which penalises large weights and reduces overfitting.

The function `_optimizer_step` (`workshops/w2/src/models/recurrent_models/train.py`) performs one update. It calls `loss.backward()`, clips the gradient norm to 1.0 to limit exploding gradients, and calls `optimizer.step()`. On GPU the forward pass runs in mixed precision (`float16`). Neither model uses dropout; overfitting is controlled with weight decay, batch normalisation and early stopping.

2.3 Hyperparameter Search

For each recurrent family, Optuna minimises log-loss on the `tune` partition using the same Tree-structured Parzen Estimator sampler, seed, five-trial budget and search space (Table 8). This paired design makes the capacity search comparable rather than favouring one cell with a larger budget.

Hyperparameter	**Search space**
`layer_dim`	Integer in $[2, 4]$
`hidden_dim`	$\{32, 64, 128\}$
Learning rate η	$[10^{-4}, 10^{-2}]$, log scale
Weight decay λ	$[10^{-6}, 10^{-2}]$, log scale
Batch size	Fixed at 512

Table 8: Shared Optuna search space for LSTM and GRU.

3 Training

3.1 Data Split

The February cohort is divided into three disjoint partitions, and the March cohort is kept entirely for the final test (Table 9):

- **Fit** (75%): trains the model.
- **Stop** (12.5%): validates the model after every epoch for early stopping.
- **Tune** (12.5%): evaluates each Optuna configuration and selects the decision threshold.

Each user is assigned to a partition by a stable hash of their `msno`, stratified by label, so the split is reproducible and the three partitions keep the same churn rate. Optuna trains each configuration on `fit`, controls overfitting with `stop` and evaluates it on `tune`. The `test` cohort takes part in no decision.

Partition	Cohort	Users	Churners	Churn rate
`fit`	February	744,699	47,604	6.39%
`stop`	February	124,117	7,934	6.39%
`tune`	February	124,115	7,933	6.39%
`test`	March	970,960	87,330	8.99%

Table 9: Partitions used for training, tuning and testing.

Before training, the numeric channels are clipped at upper quantile limits and standardised with a mean and standard deviation computed only on the days where the channel is defined (active days for listening, payment days for events, days with a known state for states); the binary flags are left unchanged. During the search these parameters are fitted only on `fit`, so no information from `stop`, `tune` or `test` enters the scaling.

3.2 Training Procedure

Each Optuna trial trains one LSTM or GRU on `fit` with batches of 512 users for at most 40 epochs. After every epoch the loss is computed on `stop`; if it does not improve for 5 consecutive epochs (*patience*), training stops and the weights of the best epoch are restored. The restored model then predicts `tune`, and its log-loss is returned to Optuna:

$$\text{log-loss} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\Big]. \tag{22}$$

We use log-loss as the main criterion because it evaluates the quality of the predicted probabilities, not only the final decision.

3.3 Optuna Results

All five trials completed for both families. The same first configuration won both searches: two recurrent layers, 32 hidden units, learning rate 2.29×10^{-3} and weight decay 5.37×10^{-4}. Table 10 reports the selected runs. The GRU obtained a slightly lower tuning log-loss (0.14653 versus 0.14674), converged one epoch earlier and required 3,840 fewer trainable parameters.

Model	**Layers**	**Hidden**	η	λ	**Params**	**Best ep.**	**Tune loss**
LSTM	2	32	2.29×10^{-3}	5.37×10^{-4}	15,457	11	0.14674
GRU	2	32	2.29×10^{-3}	5.37×10^{-4}	11,617	10	**0.14653**

Table 10: Best Optuna configuration for each recurrent family.

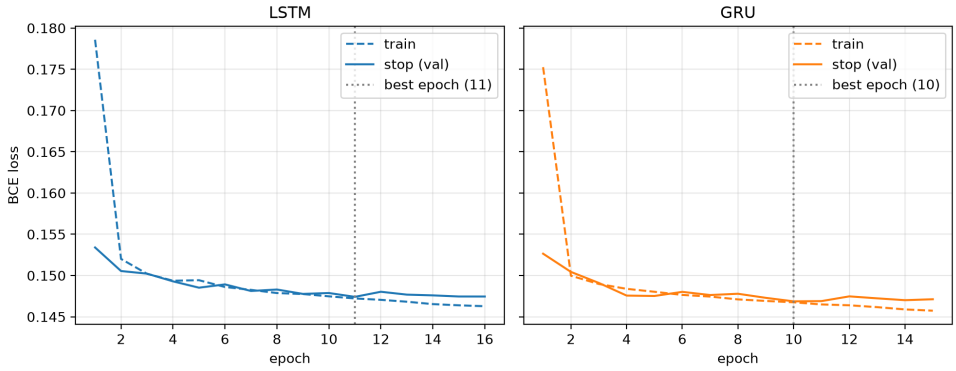


Figure 4: Training and early-stopping loss histories of the selected LSTM and GRU trials.

Decision thresholds. Each model's threshold is selected independently on `tune`, in steps of 0.01 from 0.01 to 0.99, to maximise churn-class F1. The selected thresholds are 0.27 for LSTM and 0.24 for GRU. Both are frozen before examining the test cohort.

Final models. Each selected architecture is retrained on the whole February cohort (992,931 users) for its selected number of epochs, with preprocessing refitted on all February data. Each final model then predicts March once.

4 Test

4.1 Metrics

Both final models are evaluated on the same March cohort (970,960 users, 8.99% churners). Probability metrics (log-loss and Brier score) measure probability quality; ranking metrics (ROC-AUC and average precision) measure ordering; threshold-dependent metrics describe the operating point selected on February. Table 11 reports the paired results.

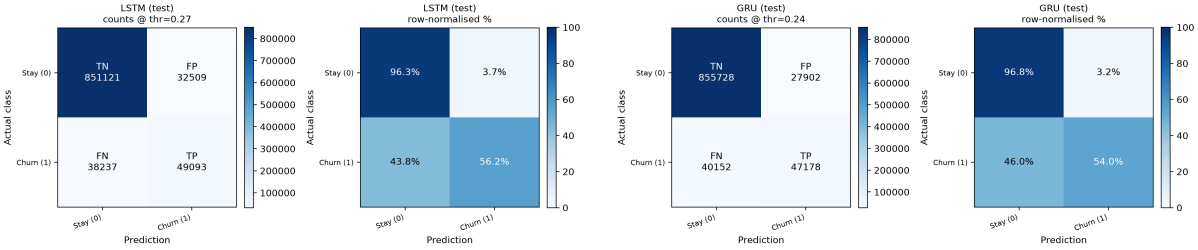
Metric	LSTM (thr. 0.27)	GRU (thr. 0.24)
Log-loss	**0.20384**	0.20707
Brier score	**0.05412**	0.05501
ROC-AUC	**0.85030**	0.84955
Average precision	0.58075	**0.58081**
Accuracy	0.92714	**0.92991**
Balanced accuracy	**0.76268**	0.75433
Precision (churn)	0.60162	**0.62837**
Recall (churn)	**0.56216**	0.54023
Specificity	0.96321	**0.96842**
F1 (churn)	**0.58122**	0.58097
F1 (macro)	0.77066	**0.77137**
MCC	0.54174	**0.54482**
Predicted churn rate	8.40%	7.73%

Table 11: Test metrics at the model-specific thresholds selected on February. Bold marks the better value for each metric.

The two models are practically tied in F1 and average precision. LSTM favours recall and probability quality, whereas GRU's lower threshold still produces fewer positive predictions and consequently higher precision, specificity, accuracy and MCC. The default 0.5 threshold is inappropriate for both imbalanced classifiers: it lowers churn F1 to 0.453 for LSTM and 0.402 for GRU.

4.2 Confusion Matrix

Figure 5 shows both confusion matrices. LSTM detects 49,093 churners (56.2%) and misses 38,237; GRU detects 47,178 (54.0%) and misses 40,152. GRU reduces false alarms among loyal users from 32,509 to 27,902, which explains its precision advantage. The operational choice is therefore explicit: LSTM finds 1,915 additional churners at the cost of 4,607 additional false positives.



(a) LSTM, threshold 0.27

(b) GRU, threshold 0.24

Figure 5: Confusion matrices on the March test cohort at the model-specific thresholds.

4.3 ROC and Precision-Recall Curves

Figure 6 overlays the curves. They are almost indistinguishable: the LSTM has a 0.00075 advantage in ROC-AUC, while the GRU has a 0.00006 advantage in average precision. Both are far above the 8.99% precision baseline of a random ranking.

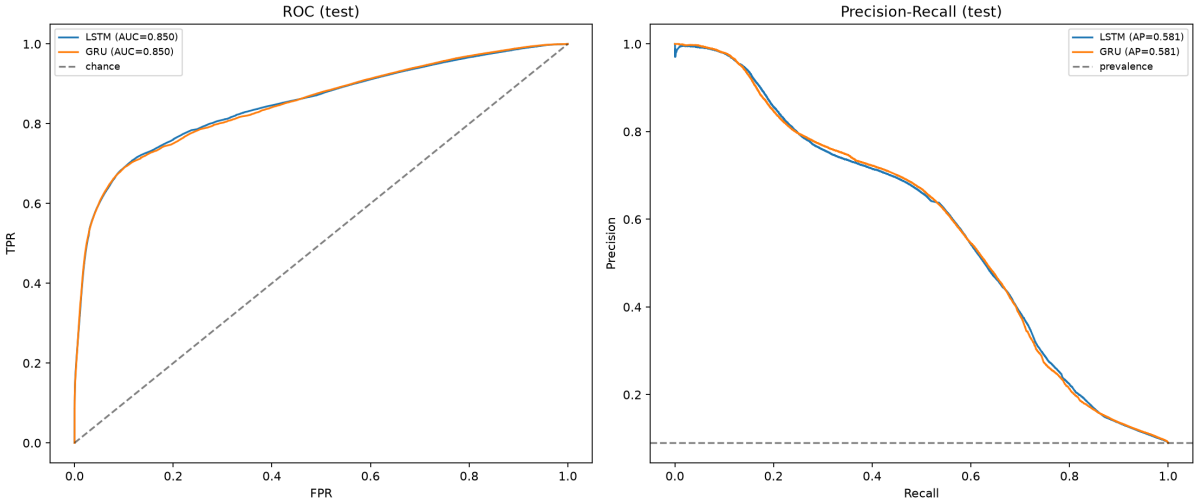
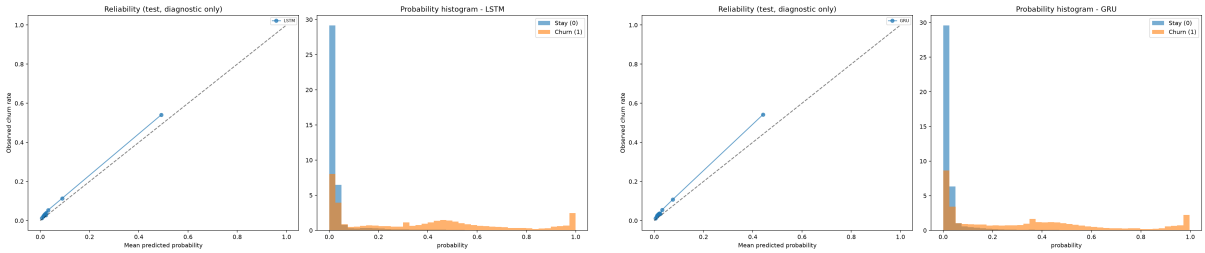


Figure 6: ROC and precision-recall curves for LSTM and GRU on the March test cohort.

4.4 Calibration

Figure 7 shows each reliability diagram and score distribution. Both models underestimate churn in March, consistent with the prevalence increase from 6.4% in February to 9.0% in March. LSTM nevertheless has the lower log-loss and Brier score, so its probabilities are better calibrated overall. Neither model was recalibrated after the temporal shift.



(a) LSTM (b) GRU

Figure 7: Reliability diagrams and predicted-probability distributions on the March test cohort.

4.5 Statistical and Operational Comparison

Because a dataset of almost one million users can make very small differences statistically detectable, we report paired 95% bootstrap confidence intervals and interpret effect size as well as significance (Table 12). DeLong's paired test also detects the LSTM's ROC-AUC advantage ($\Delta = 0.00075$, $z = 2.47$, $p = 0.0134$). However, the magnitude is operationally negligible. The bootstrap intervals show no reliable difference in average precision or F1. The clear difference is log-loss: LSTM improves it by 0.00323, although the practical gain remains modest.

Metric	**LSTM (95% CI)**	**GRU (95% CI)**	**Δ LSTM–GRU (95% CI)**	**Sig.**
ROC-AUC	0.8503 [0.8487, 0.8519]	0.8495 [0.8480, 0.8511]	0.00075 [0.0001, 0.0014]	Yes
Average precision	0.5808 [0.5774, 0.5840]	0.5808 [0.5774, 0.5842]	−0.00006 [−0.0009, 0.0006]	No
F1 (churn)	0.5812 [0.5783, 0.5839]	0.5810 [0.5780, 0.5837]	0.00024 [−0.0009, 0.0014]	No
Log-loss	0.2038 [0.2026, 0.2051]	0.2071 [0.2058, 0.2084]	−0.00323 [−0.0034, −0.0030]	Yes

Table 12: Paired bootstrap comparison on March. For log-loss, a negative difference favours LSTM.

The models also produce nearly identical ranked contact lists (Figure 8). If the retention team contacts the top 5% of users, LSTM captures 39.82% of all churners (lift 7.96) and GRU captures 40.14% (lift 8.03). At 10%, they capture 60.17% and 60.32%, respectively. LSTM becomes slightly better at 20% and 30% contact budgets. Thus GRU is marginally preferable for a very narrow campaign, but the difference is less than 0.4 percentage points.

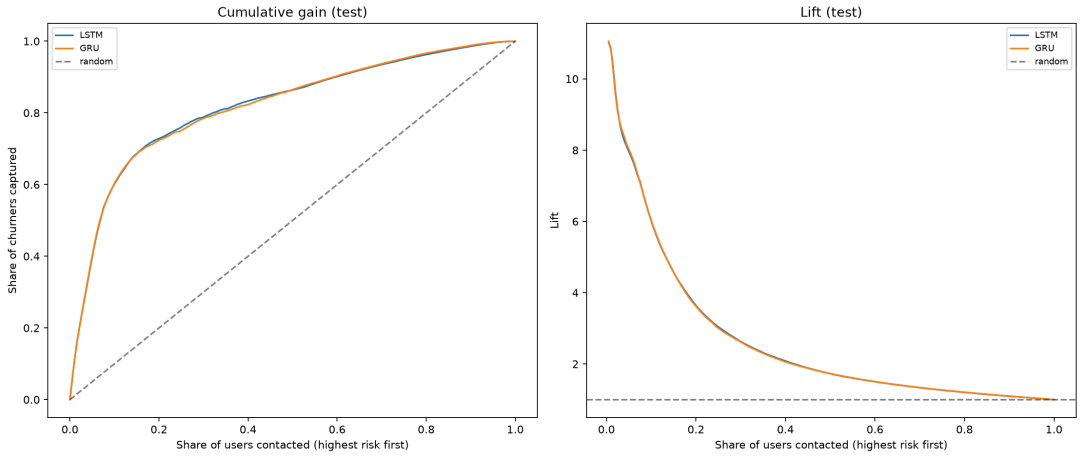


Figure 8: Cumulative gain and lift for ranked retention campaigns.

Efficiency. At the selected architecture, GRU uses 11,617 parameters versus 15,457 for LSTM, a 24.8% reduction, and takes 16.62 versus 18.11 seconds per epoch, an 8.2% reduction in the recorded run. Given their virtually equal ranking and classification performance, GRU offers the better efficiency–performance trade-off; LSTM is preferable when probability quality or maximum recall is the priority.

4.6 Comparison with Workshop 1

Table 13 adds both recurrent models to the three tabular models from Workshop 1. All are evaluated on March after choosing thresholds on February. The tabular models use 59 engineered features, demographics and complete transaction history; LSTM and GRU use only the 90-day sequence.

Model	Thr.	Log-loss	Brier	AUC	AP	Prec.	Recall	F1	MCC
Logistic reg. (W1)	0.28	0.2126	0.0564	0.8445	0.5553	0.5811	0.5343	0.5567	0.5155
Random forest (W1)	0.26	**0.2032**	0.0548	**0.8632**	**0.5933**	**0.6379**	0.5385	**0.5840**	**0.5490**
XGBoost (W1)	0.88	0.4886	0.1498	0.8520	0.5882	0.6094	**0.5424**	0.5740	0.5356
LSTM (W2)	0.27	0.2038	**0.0541**	0.8503	0.5808	0.6016	**0.5622**	0.5812	0.5417
GRU (W2)	0.24	0.2071	0.0550	0.8495	0.5808	0.6284	0.5402	0.5810	0.5448

Table 13: Test metrics on March for Workshop 1 tabular models and Workshop 2 recurrent models. Best value per column is bold.

Random forest retains the best ROC-AUC, average precision, F1 and MCC, but LSTM now has the best Brier score and recall and is only 0.0006 worse in log-loss. Both recurrent models outperform logistic regression in ranking and F1, and greatly outperform the poorly calibrated Workshop 1 XGBoost in log-loss. The comparison also shows that cell choice is much less important than representation: LSTM and GRU differ from each other far less than either differs from the tabular models.

5 Conclusions

LSTM versus GRU. Both gated recurrent cells extract almost the same predictive signal from the daily sequence: F1 differs by only 0.00024 and average precision by 0.00006. LSTM yields better probabilities and detects more churners, while GRU produces fewer false alarms, uses 24.8% fewer parameters and trains 8.2% faster per epoch. Therefore there is no universally superior cell: LSTM is the choice for probability quality and recall, whereas GRU is the parsimonious choice when compute, model size or contact precision matters.

Relation with Workshop 1. Workshop 1 summarises each subscriber into 59 hand-crafted features, while both recurrent models learn from a 90×20 daily sequence. Under the same temporal protocol, LSTM and GRU approach the random forest (F1 0.581 versus 0.584) without demographics or full-history aggregates. This supports the value of temporal order, but the remaining ranking gap suggests that tenure, demographics and older payment history still add information beyond the 90-day window.

How the sequence captures behaviour. The daily tensor preserves patterns compressed by monthly aggregates: a user who stops listening during the last two weeks, cancellation followed by inactivity, or expiry without renewal. Both cells can preserve or discard earlier information through gates; LSTM uses separate memory and hidden states, whereas GRU performs the same role with a single state. We did not attribute predictions to individual days or channels, so claims about which pattern each network actually uses require a future interpretability analysis.

Why no further experiments are possible. The main limitation is the dataset itself. The competition publishes official churn labels for only two months: February 2017 (`train.csv`) and March 2017 (`train_v2.csv`). A user is labelled as churn when their membership expires in the target month and they do not subscribe again within the following 30 days, so labelling a month requires the transactions of the next 30 days. Since the published transactions and logs end on 2017-03-31, no later month (April 2017 onwards) can be labelled. Earlier months, such as January 2017 or December 2016, could in principle be labelled by applying the official labelling rule to the transactions, and the listening history before the window is available since 2015. However, our reconstruction of that rule, a port of the official `WSDMChurnLabeller.scala`, agrees with the official labels for only 98.8% of the February users and 97.0% of the March users, and about 137,000 official users per month cannot be reconstructed from the published transactions. Labels for other months would therefore be noisier than the official ones and would not define the same cohorts, so a model trained or tested on them would not be comparable with the results reported here. For this reason the evaluation is limited to one training month and one test month, and we cannot measure how stable the model is across several months.

References

[1] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical evaluation of gated recurrent neural networks on sequence modeling.* arXiv:1412.3555. https://doi.org/10.48550/arXiv.1412.3555

[2] KKBOX. (2017). *WSDM–KKBox's Churn Prediction Challenge* [Data set and competition]. Kaggle. https://www.kaggle.com/competitions/kkbox-churn-prediction-challenge